LAB2：神经网络如何学习不同频率的信号

中国科学技术大学

2026 年秋季学期　学生版
截止时间：2026 年 11 月 23 日 06:00

个人完成，满分 100 分。使用 NumPy 手写梯度进行核验，使用 PyTorch 完成主实验；CPU 即可，无需 GPU。

提交与评分

提交两项：报告`学号-姓名-LAB2.pdf` 上传至 Blackboard（bb.ustc.edu.cn），单个 `submission.py` 上传至 OJ（oj.temaurinum.moe）。不接收其他文件。报告建议 6–8 页；图表与运行记录直接放入报告。允许 AI 辅助，但须说明使用方式并给出独立核验，所有提交内容须能本人解释。

类别	分值	内容
OJ 代码	40	Fourier 映射 8；NumPy 前向 6；损失与反向 14；PyTorch 网络与参数对应 6；PyTorch 目标 6
OJ 最终性能	10	未公开信号实例上的统一重训与预测
报告	50	实现与三方核验 8；频率分析 12；联合选参 14；反例 10；二维迁移 4；复现与独立核验 2
环境创建	0	独立环境与检查，不计分

报告由下述六项任务组成，各项分值合计为 $8+12+14+10+4+2=50$ 分。

1　环境与文件（不计分）

新建 `ai3002-lab2` 环境，不复用 LAB0/LAB1 环境。若尚未安装 Conda，请先按 LAB0 安装 Miniforge。

Windows 打开 PowerShell，进入实际保存的 LAB2 根目录；若仓库位于 LAB0 的示例位置，可执行第一行，否则自行替换路径。

```
cd $HOME\ALML_public\LABs\LAB2
conda env create -f environment.yml
conda activate ai3002-lab2
python src/verify_env.py
```

macOS 打开 Terminal，进入实际目录后执行：

```
cd ~/ALML_public/LABs/LAB2
conda env create -f environment.yml
conda activate ai3002-lab2
python src/verify_env.py
```

若 Shell 尚不能激活环境，Windows 先在 Miniforge Prompt 执行 `conda init powershell`，macOS 执行 `conda init zsh`，然后关闭并重开终端。在 VS Code 中运行 “Python: Select Interpreter”，选择本实验环境。以后每次运行前只需激活，不必重新创建。环境已有同名环境时先检查其来源，不要直接删除个人环境。

检查成功显示 `LAB2 environment ready`，并保存 `results/environment.json`。确认其中解释器路径属于本实验环境。所有下述命令均从 LAB2 根目录运行；所有代码位于 `src/`，数据位于 `data/`，输出位于 `results/`。只修改 `src/submission.py`。代码、模型、运行日志和数据均不作为报告以外的额外附件提交。

2 数据、网络与统一训练规则

2.1 数据及用途

三个公开实例各有 256 个训练点和 256 个验证点，输入位于 $[-1, 1)$。每个信号的形式为

$$f(x) = \sum_{r=1}^{4} a_r \sin(\pi k_r x + \theta_r), \qquad (a_1, a_2, a_3, a_4) = \frac{(1, 0.5, 0.3, 0.2)}{\sqrt{(1^2 + 0.5^2 + 0.3^2 + 0.2^2)/2}}.$$

每个 k_r 分别从以下四组中抽取，θ_r 为随机相位：

$$\{1, 2\}, \quad \{3, 4, 5\}, \quad \{7, 8, 9, 10\}, \quad \{13, 14, 15, 16, 17, 18\}.$$

数据中的 `freqs` 记录该实例的四个频率。`X_train,y_train` 用于训练，`X_val,y_val` 用于选参。`X_grid,y_grid` 是与二者不重合的 2048 点等距诊断网格，只用于画图、分析残差。诊断网格不是独立最终测试集，不得用其误差排名选参。不能在实现中直接编写目标函数、读取数据文件或猜测未公开相位。

另外提供 32 点稀疏训练数据，其标签加有标准差为 0.15 的独立高斯噪声，以及一张 64×64 的课程生成灰度图。图像包含平滑区域、边缘与条纹，固定选出 1024 个观测像素；其余像素仅用于最后一次迁移评估。

2.2 输入表示和网络

对一维输入，Fourier 特征按以下顺序排列：

$$\phi_s(x) = \left[x, \sin(\pi s 2^0 x), \ldots, \sin(\pi s 2^5 x), \cos(\pi s 2^0 x), \ldots, \cos(\pi s 2^5 x)\right].$$

二维时对每个坐标分别生成 13 维向量，再按坐标顺序拼接为 26 维。$s > 0$ 为频率尺度，不是需要求梯度的模型参数。

设批量输入特征为 $F \in \mathbb{R}^{n\times d}$，标签为 $y \in \mathbb{R}^{n\times 1}$。网络为

$$\begin{aligned} A_1 &= FW_1 + \mathbb{1}_n b_1^\top, \quad H_1 = \mathrm{ReLU}(A_1), \\ A_2 &= H_1W_2 + \mathbb{1}_n b_2^\top, \quad H_2 = \mathrm{ReLU}(A_2), \\ \widehat{y} &= H_2W_3 + \mathbb{1}_n b_3^\top. \end{aligned}$$

这里 n 为批量大小，d 为特征维度，$\mathbb{1}_n$ 为全 1 列向量；$\mathrm{ReLU}(t) = \max(0, t)$ 逐元素作用，零点导数在本实验定义为 0。主实验两个隐藏层均为 64 维，因此

$$W_1 \in \mathbb{R}^{d\times 64}, \quad W_2 \in \mathbb{R}^{64\times 64}, \quad W_3 \in \mathbb{R}^{64\times 1}, \quad b_1, b_2 \in \mathbb{R}^{64}, \quad b_3 \in \mathbb{R}^1.$$

目标函数使用均方误差，注意没有额外的二分之一：

$$J = \frac{1}{n}\sum_{i=1}^{n}(\widehat{y}_i - y_i)^2 + \frac{\lambda}{2}\sum_{\ell=1}^{3}\|W_\ell\|_F^2.$$

其中 $\|W\|_F^2$ 表示全部矩阵元素的平方和，$\lambda \geq 0$；偏置不受惩罚。

2.3 训练协议（不得改变）

统一使用全批次 Adam，学习率为 η，其余参数为

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}, \quad \text{weight decay} = 0.$$

每次从课程指定初始化出发更新 800 次，采用最后一次参数；不提前停止，不热启动，不使用 AdamW。随机种子为 20261021；前两层权重方差为输入维数倒数的两倍，输出层为输入维数的倒数，偏置为零。课程提供初始化数组；PyTorch 必须原样装入。

主训练使用单线程 CPU 和单精度；梯度核验使用双精度。另提供宽度为 70 的原始坐标模型作为近似匹配的对照。

3 任务一：实现与三方核验（报告 8 分）

补全 `submission.py` 的五个计算接口，并在最终选参后填写配置接口。NumPy 部分不得调用 PyTorch 或自动微分；PyTorch 部分允许自动微分。预测及标签统一为二维列数组，不能混用 `(n,)` 与 `(n,1)` 导致广播。

接口	要求
`fourier_features(x, scale)`	支持 NumPy 数组和 PyTorch Tensor，返回同后端、同精度、同设备的特征。Tensor 版本保持对输入的自动微分。
`numpy_forward(x, params)`	返回预测及自定义缓存。参数键为 `W1,b1,W2,b2,W3,b3`；权重按“输入维数乘输出维数”存储。
`numpy_loss_backward(...)`	接收预测、标签、缓存、参数和正则化系数，返回标量目标及全部六个参数梯度。不得修改输入。
`build_torch_model(params)`	返回 `Sequential(Linear,ReLU,Linear,ReLU,Linear)`，按给定参数尺寸创建并原样载入。`Linear.weight` 需要转置。
`torch_objective(...)`	返回与 NumPy 相同且可微的标量目标。

`get_config()`	返回固定的 `scale,lr,lam` 三个数。

```
python src/run.py check
```

公开检查使用远离 ReLU 零点的小网络，在相同参数与输入下，对零和非零正则化分别进行以下两组比较。记当前参数数组的 NumPy 解析梯度、PyTorch 自动微分梯度、中心有限差分梯度分别为 g_{np}、g_{pt}、g_{num}。

对单个参数坐标 u，保持其余参数不变，中心有限差分为

$$g_{\mathrm{num}}(u) = \frac{J(u+\varepsilon) - J(u-\varepsilon)}{2\varepsilon}, \qquad \varepsilon = 10^{-6}.$$

第一组：NumPy 解析梯度与中心有限差分。每个参数数组的归一化误差须满足

$$E_{\mathrm{num,np}} = \frac{\max |g_{\mathrm{num}} - g_{\mathrm{np}}|}{\max\{1, \max |g_{\mathrm{num}}|, \max |g_{\mathrm{np}}|\}} < 10^{-6}.$$

这里的最大值均遍历当前参数数组的全部元素。

第二组：PyTorch 自动微分梯度与 NumPy 解析梯度。脚本使用 `np.allclose` 逐元素检查，要求对每个元素索引 i 都有

$$|g_{\mathrm{pt},i} - g_{\mathrm{np},i}| \le 10^{-10} + 10^{-9}|g_{\mathrm{np},i}|.$$

其中绝对容差为 10^{-10}，相对容差为 10^{-9}；权重梯度先转置为 NumPy 的布局再比较。

输出说明：`check.json` 中的 `relative_errors` 保存第一组的归一化误差。第二组通过断言检查，不单独保存误差数值；检查成功时 `passed=true` 表示两组均通过，第二组失败时会报 `torch mismatch`。

报告要求：列出两种正则化下六组参数的第一组归一化误差，并记录第二组是否通过；说明为何检查点应避开 ReLU 零点。若修复过错误，给出一项错误、修正和核验记录；若首次通过，给出对批量平均或偏置惩罚的一项独立核验。

4 任务二：观察频率学习过程（报告 12 分）

```
python src/run.py observe
```

脚本对三个公开实例运行原始坐标宽度 64、原始坐标宽度 70、Fourier 宽度 64 三个对照；固定 $s = 1$、$\eta = 10^{-3}$、$\lambda = 0$。记录第 0、50、200、800 次更新的预测，并每 50 次更新记录训练、验证及诊断误差。

对等距诊断点 x_j、残差 $r_j = \widehat{y}(x_j) - f(x_j)$，工具计算

$$\alpha_k = \frac{2}{M}\sum_{j=1}^{M} r_j \sin(\pi k x_j), \quad \beta_k = \frac{2}{M}\sum_{j=1}^{M} r_j \cos(\pi k x_j), \quad E_k = \frac{\alpha_k^2 + \beta_k^2}{2}.$$

这里 $M = 2048$，k 为该实例的一个目标频率。E_k 衡量残差在该频率上的分量；只检查四个频率并不能覆盖全部误差，其和不必等于 MSE。

报告要求：选择一个实例展示三个对照的学习曲线，并汇总另两个实例是否支持相同结论；结合初始幅度，使用绝对误差或初值归一化误差说明哪些分量下降更快。讨论近似参数量匹配后结论是否改变。

5 任务三：联合选参（报告 14 分）

先运行六组基础对照：

```
python src/run.py tune
```

尺度取 $\{0.25, 1, 4\}$，学习率取 $\{10^{-3}, 3 \times 10^{-3}\}$，正则化为 0。每个组合分别在三个实例训练，以三个验证 MSE 的平均值选出最佳组合。先保存并阅读这一轮结果，再在该最佳尺度、学习率下补选两个不同的正则化系数，范围为 $[10^{-6}, 10^{-1}]$。例如下面命令：

```
python src/run.py tune --extra-lambdas 0.0001 0.01 --reason "Compare weak and strong penalties after
    reading coarse results"
```

将自己的观察、补选理由和实际命令写入报告；选值理由必须形成于查看补选结果之前。最终从六组基础与两组补选中，按平均验证 MSE 最小选择。相同时依次按尺度、学习率、正则化系数从小到大打破平局。脚本会输出 `recommended_config`。补选阶段重新执行时保留最新两组，报告应如实说明曾进行的其他尝试。

报告要求：给出八行完整对照表，包括参数、训练是否成功、三个实例的验证误差和均值；说明为什么选这些补选值、最终如何选择、是否存在尺度过大却训练误差很低的情形。失败或未改善试验保留，不扣除其方法和分析部分的分数。不得根据密集网格、二维未观测像素或 OJ 结果反向调参。

6 任务四：稀疏含噪反例（报告 10 分）

```
python src/run.py stress
```

在三个实例上使用提供的 32 点含噪训练数据。学习率固定为任务三选出的值，比较三种条件：中等尺度 $s = 1$、高尺度 $s = 4$、高尺度加正则化。前两者正则化为零。

第三种条件优先使用最终所选正则化。若最终配置为零正则化，则使用两个正补选值中平均验证误差较小者。不得根据反例结果再次选择正则化。

报告要求：汇总三种条件的含噪训练误差、干净验证误差和干净诊断误差；展示一个代表实例的预测图。回答：训练误差更小能否推出恢复干净函数更好？正则化是否在所有实例均有改善？哪些结论可由这些控制变量的对照支持，哪些仍不能？

7 任务五：二维迁移（报告 4 分）

```
python src/run.py transfer
```

将最终所选尺度、学习率与正则化直接用于图像坐标回归，隐藏层宽度仍为 64、训练仍为 800 次。只在 1024 个观测像素上训练，不读取其他像素作为训练标签。此阶段只作一次迁移评估，不安排新的参数搜索。

报告要求：放入原图、观测掩码、重建图、绝对误差图，报告未观测像素 MSE；用简短文字指出误差主要出现于平滑区域、边缘还是条纹，并说明一维最优配置为何不保证二维最优。

8 任务六：复现与独立核验（报告 2 分）

将 `results/tune.json` 中的 `recommended_config` 原样填入 `submission.py` 的 `get_config()`，然后执行：

```
python src/run.py final
```

脚本核对配置与记录一致，重新训练并检查验证结果可复现。修改 `get_config()` 不会使实现核验失效；修改其他计算代码后须重新运行各阶段。数据校验失败应恢复课程数据，不应通过修改校验文件绕过。最终核验保存代码摘要、种子、配置、环境与耗时，供报告复核。

阶段	主要输出	报告用途
check	`check.json`	六组参数的三方梯度核验
observe	`observe.json`；每个实例的 `history.json`，`predictions.png`，`frequencies.png`	频率学习过程与参数量对照
tune	`tune_coarse.json`，`tune.json`，`trials.csv`	粗比较、补选理由与最终八组结果
stress	`stress.json` 及对应预测图	稀疏含噪反例
transfer	`transfer.json`，`transfer.png`	二维未观测像素误差与图像
final	`final.json`	最终配置、重训一致性、实现摘要

这些文件仅本地留存。**报告要求（共 2 分）：**

- **复现记录（1 分）**：在报告末尾记录软件版本、设备、完整运行命令、最终配置与总用时，使他人能按记录复现实验。
- **独立核验记录（1 分）**：说明 AI 使用情况，并给出一项可复核的自检过程与结果；未使用 AI 也应给出自检。可引用任务一已有的独立核验记录，无需额外增加实验。

问题通过课程仓库 Issues提出，注明 LAB2 和任务编号，不公开完整实现或个人信息。

最终性能的简要规则

OJ 使用八个未公开的一维信号实例，按统一 CPU 协议重新训练；你的配置在全部实例固定。参考基准使用相同架构与预算，固定 $s = 0.25$、$\eta = 3 \times 10^{-3}$、$\lambda = 0$。汇总全部未见位置的 RMSE；不劣于基准得 10 分，否则按基准 RMSE 与提交 RMSE 的比值折算。训练失败或非有限输出时性能项为零，代码分独立评测。二维任务只计报告分。

只能提交通用计算实现，不能提交预训练权重、硬编码信号、读取外部数据或改变训练协议。最终性能仅占 10 分，报告按证据、方法和解释评分，未获得最好的单次曲线不妨碍获得分析分。